CUSTOMER CHURN PREDICTION

End-to-End Machine Learning Project Report

Project objective
Estimate customer churn probability from historical customer and service data, then make the model available through an interactive Streamlit application.

Project detail	Implementation
Dataset	IBM Telco Customer Churn
Model	Logistic Regression
Training environment	Google Colab
Application development	VS Code + Streamlit
Model artifact	customer_churn_model.joblib
Version control	GitHub
Cloud hosting	Streamlit Community Cloud — deployment pending

Project status

Completed: data preparation, model training, validation-based threshold selection, held-out test evaluation, model packaging, local Streamlit application, and GitHub file upload.

Remaining: complete the Streamlit Community Cloud deployment and verify the public application URL.

Prepared as a portfolio project report

1. Executive Summary

Customer churn is a significant business problem for subscription-based companies. When customers discontinue service, businesses can lose recurring revenue and incur additional customer acquisition costs.

This project developed a binary classification model to estimate customer churn probability using the IBM Telco Customer Churn dataset. The workflow covered data exploration, cleaning, feature preprocessing, model training, threshold selection, final evaluation, model serialization, and application development.

A Logistic Regression model was trained in Google Colab. The final threshold was selected on a validation dataset, and the model was evaluated on a separate test dataset. The trained model and fitted preprocessing pipeline were saved in a Joblib artifact and reused in a Streamlit application built in VS Code.

The local app successfully produced a sample prediction. Cloud deployment through Streamlit Community Cloud is the remaining milestone.

Final held-out test metrics

Metric	Result
Accuracy	76.44%
Precision	54.04%
Recall	75.13%
F1-score	62.86%
ROC-AUC	0.8465
Selected threshold	0.30

2. Business Problem and Objective

Problem statement: How can a subscription-based business identify customers who may be at risk of leaving so its retention team can prioritize engagement?

The application estimates churn probability and flags customers whose predicted probability meets or exceeds a selected threshold. This can support prioritization, but predictions should not be treated as certain outcomes.

- Identify customers who may require retention attention.
- Provide a probability-based signal rather than a simple unqualified guess.
- Demonstrate an end-to-end machine learning workflow that can be used through a web interface.

The threshold and deployment design should ultimately reflect the business cost of missed churners, false positives, and retention interventions.

3. Tools and Technologies

Technology	Purpose
Python	Programming language
Pandas / NumPy	Data loading, cleaning, and numerical operations
Scikit-learn	Preprocessing, Logistic Regression, and evaluation
StandardScaler	Scale numerical features
OneHotEncoder	Encode categorical features
ColumnTransformer	Combine feature-specific preprocessing
Joblib	Serialize and load the model artifact
Google Colab	Exploration and model training
VS Code	Application development
Streamlit	Interactive web interface
GitHub	Source code and model artifact repository

4. Dataset Exploration

The IBM Telco Customer Churn dataset contained 7,043 records and 21 columns. The target variable was `Churn`, while `customerID` was used as an identifier and excluded from model training.

4.1 Target distribution

Outcome	Customers	Share
No churn	5,174	73.46%
Churn	1,869	26.54%
Total	7,043	100%

The target was imbalanced: customers who did not churn represented about 73.46% of the dataset. Therefore, accuracy alone would not be sufficient to assess churn detection.

4.2 Exploratory observations

Group	Average tenure	Median tenure
Did not churn	37.57 months	38 months
Churned	17.98 months	10 months

Observed churn rates by contract type

Contract type	Observed churn rate
Month-to-month	42.71%
One year	11.27%
Two year	2.83%

These are descriptive associations in this dataset; they do not prove that tenure or contract type independently causes churn.

5. Data Format and Ingestion Practice

Additional exercises helped practise working with common data formats and external data sources. The churn dataset was converted to Excel, JSON, and Parquet, then reloaded for consistency checks.

A separate API exercise retrieved 100 sample posts from JSONPlaceholder, converted them to a DataFrame, saved them to CSV, and reloaded them. The comparison returned True. A separate UCI Iris download exercise produced a DataFrame with 150 records and five columns.

Scope note: The API posts and Iris dataset were independent learning exercises. They were not combined with the churn dataset and were not used to train the churn model.

6. Data Cleaning and Feature Setup

6.1 TotalCharges

The `TotalCharges` column was initially stored as an object. Conversion to numeric identified 11 blank or invalid values. All 11 records had zero tenure, a populated monthly charge, and Churn = No. These values were filled with zero, and the column became float64 with no missing values.

6.2 Features and target

The feature matrix contained 19 inputs and the target vector contained 7,043 labels. The target was mapped as No → 0 and Yes → 1. The identifier `customerID` was excluded.

Numerical features (4)

SeniorCitizen, tenure, MonthlyCharges, TotalCharges

Categorical features (15)

gender, Partner, Dependents, PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies, Contract, PaperlessBilling, PaymentMethod

7. Training, Validation, and Test Strategy

An initial stratified 80/20 split was used for baseline experimentation. During that experiment, threshold comparisons were made on the test data, so those results were treated as exploratory rather than as the final unbiased evaluation.

The workflow was then corrected to use a stratified 60/20/20 split.

Dataset	Records	Purpose
Training	4,225	Fit preprocessing and train the model
Validation	1,409	Compare thresholds and select one
Test	1,409	Evaluate final model
Total	7,043	Full dataset

The churn rate remained approximately 26.54% across the splits. This design separates model fitting, threshold selection, and final evaluation.

8. Feature Preprocessing

Numerical features were standardized using StandardScaler. Categorical features were transformed using OneHotEncoder configured with handle_unknown='ignore'. Both transformations were combined in a ColumnTransformer.

The preprocessor was fitted only on the training data. The fitted transformer was then used to transform validation and test data. This avoids fitting preprocessing steps on held-out data and helps prevent leakage.

The transformed feature matrices contained 45 columns.

9. Model Training

Logistic Regression was selected as the baseline binary classifier. The final model used max_iter=2000 and random_state=42 and trained successfully in 24 iterations without convergence warnings.

Logistic Regression provides predicted probabilities and supports coefficient inspection. Coefficients represent conditional associations in the fitted model, not proof of causation. Correlated features can also complicate interpretation.

10. Baseline Threshold Experiments

The initial baseline model achieved approximately 80.55% accuracy at a threshold of 0.50, with ROC-AUC around 0.8421. Threshold experiments showed how precision and recall change as the cutoff changes.

Threshold	Precision	Recall	F1-score
0.30	51.93%	75.40%	61.50%
0.40	56.82%	66.84%	61.43%
0.50	65.72%	55.88%	60.40%

Because the initial test set was used during threshold exploration, these baseline results were not used as the final performance claim.

11. Correct Threshold Selection

A fresh model was trained using the corrected 60/20/20 split. Thresholds were compared on the validation dataset, and 0.30 was selected to prioritize recall while considering F1-score.

Threshold	Precision	Recall	F1-score
0.30	53.89%	75.94%	63.04%
0.40	58.80%	65.24%	61.85%
0.50	64.82%	53.21%	58.44%

The lower threshold identifies more potential churners but also produces more false positives. The final threshold should ultimately be assessed against business costs and operational capacity.

12. Final Test Evaluation

After selecting the threshold on validation data, the final model was evaluated on the separate held-out test dataset.

Metric	Final test result	Interpretation
Accuracy	76.44%	Share of all test predictions that were correct
Precision	54.04%	Share of flagged churn predictions that were actual churners
Recall	75.13%	Share of actual churners identified
F1-score	62.86%	Harmonic mean of precision and recall
ROC-AUC	0.8465	Ability to rank churners above non-churners across thresholds

12.1 Confusion matrix

Actual / Predicted	Predicted No Churn	Predicted Churn
Actual No Churn	796 (TN)	239 (FP)
Actual Churn	93 (FN)	281 (TP)

The model correctly identified 281 of the 374 actual churners in the test set and missed 93. It also flagged 239 customers who did not churn. This illustrates the trade-off between detecting more churners and generating false alarms.

13. Saving the Trained Model

The fitted preprocessing pipeline, trained Logistic Regression model, selected threshold, and feature lists were saved together in `customer_churn_model.joblib`. The artifact was reloaded and verified in Colab, then downloaded for local deployment.

Packaging these components together helps ensure the application uses the same preprocessing logic and threshold as the training workflow.

14. Building the Streamlit Application

The file `app.py` was created in VS Code. It loads the saved artifact and presents an interactive form for all 19 input features, using number inputs for numerical fields and dropdowns for categorical fields.

When a user submits the form, the application builds a one-row DataFrame, orders the columns according to the saved feature lists, applies the fitted preprocessor, obtains a churn probability, and compares it with the 0.30 threshold. The application displays the probability and risk classification.

15. Resolving the Compatibility Issue

The first local launch failed while loading the saved ColumnTransformer with an AttributeError involving the internal name _RemainderColsList.

Environment	Version
Training scikit-learn	1.6.1
Initial local scikit-learn	1.8.0
Initial local Python	3.14
Working local Python	3.12.13
Working local scikit-learn	1.6.1

Installing scikit-learn 1.6.1 directly into the Python 3.14 environment failed because pip attempted a source build and the required compiler was unavailable. Python 3.12.13 was already installed, so a separate .venv environment was created with that interpreter.

After activating the virtual environment and installing Streamlit, Pandas, scikit-learn 1.6.1, and Joblib, the model artifact loaded successfully. This resolved the compatibility issue without retraining the model.

16. Local Application Testing

The application was launched from CMD using the activated virtual environment with the command:

```
python -m streamlit run app.py
```

The browser application ran successfully and generated the following sample prediction using the default customer details:

Item	Result
Predicted churn probability	19.42%
Classification threshold	30%
Application classification	Lower Risk

Because 19.42% is below the 30% threshold, the sample customer was classified as lower risk. This confirms that the application can load the model, accept input, preprocess the customer details, calculate a probability, and display a classification. It is a sample application result, not an additional test-set metric.

17. Preparing for Cloud Deployment

The project files were organized as follows:

```
customer-churn-prediction/
■■■ app.py
■■■ customer_churn_model.joblib
■■■ requirements.txt
■■■ README.md
```

The requirements file specifies Streamlit, Pandas, scikit-learn==1.6.1, and Joblib. Pinning scikit-learn helps preserve compatibility with the saved artifact.

A GitHub repository named customer-churn-prediction was created, and app.py, customer_churn_model.joblib, and requirements.txt were uploaded. A README was prepared to document the project and local setup.

17.1 Remaining cloud deployment steps

- Sign in to Streamlit Community Cloud using GitHub.
- Select the customer-churn-prediction repository, branch, and app.py entry point.
- Create the application and inspect build logs if necessary.
- Test the hosted app using multiple customer profiles.
- Add the live application URL to the GitHub README.

Cloud deployment has not yet been confirmed. The application is currently verified as running locally.

18. Limitations and Future Improvements

- Compare Logistic Regression with Random Forest and gradient-boosted tree models.
- Evaluate probability calibration and stability across different data splits.
- Estimate the financial costs of false positives and false negatives.
- Choose the operational threshold using customer lifetime value, intervention cost, and team capacity.
- Add input validation, error handling, and batch prediction capabilities.
- Monitor prediction behavior and model performance after deployment.
- Treat the Joblib artifact as trusted executable serialization and only load it from a trusted source.

19. Key Learning Outcomes

- Data exploration, cleaning, and format conversion.
- Class imbalance and appropriate classification metrics.
- Feature separation and numerical/categorical preprocessing.
- Prevention of preprocessing leakage through train-only fitting.
- Validation-based threshold selection and held-out test evaluation.
- Model serialization and reuse in an application.
- Streamlit UI development and local deployment.
- Python virtual environments and dependency version compatibility.
- GitHub repository preparation for cloud hosting.

20. Conclusion

This project demonstrates an end-to-end machine learning workflow for customer churn prediction. Starting from a raw dataset, I explored and cleaned the data, built a preprocessing pipeline, trained Logistic Regression, selected a threshold using validation data, and evaluated the final model on an untouched test set.

The final held-out test results were 76.44% accuracy, 54.04% precision, 75.13% recall, 62.86% F1-score, and 0.8465 ROC-AUC. The saved model was packaged with its preprocessing pipeline and deployed locally through a working Streamlit interface.

The project files are on GitHub. The final milestone is to complete the Streamlit Community Cloud deployment and verify the public URL. The central achievement is not just training a model, but moving it through evaluation, packaging, dependency management, and into a functioning application.